VISVESVARAYA TECHNOLOGICAL UNIVERSITY

Jnana Sangama, Machhe, Belagavi – 590018, Karnataka

2022 SCHEME – BCS714D

ASSIGNMENT 2 REPORT

ON

Food and Performance Explorer: A Flask Dashboard for USDA FoodData Central Analytics

Track B – Part 2: Web Dashboard and Visualisation

Submitted as part of the coursework for Big Data Analytics

Sl. No.	Student name	USN	Marks	Signature
1	Abhishek R	1JS23CS004		
2	Adithya V	1JS23CS005		
3	Dhanush Singh G	1JS23CS048		

Group No.: 18 | Section: CSE A

Course Faculty: Sandhiya B

Department of Computer Science and Engineering

JSS ACADEMY OF TECHNICAL EDUCATION

JSS Campus Road, Dr. Vishnuvardhan Road, Bengaluru – 560060

Academic Year: 2026–2027 (ODD Semester)

Contents

Section	Page
1. Introduction and Dataset	3
2. Dashboard Architecture	4
3. Workflow Controls and Progress	5
4. Theme-Based Result Presentation	6
5. Evaluation and Recorded Results	7
6. Conclusion and References	8

Dataset: USDA FoodData Central, October 31, 2024 CSV release. Group 18, CSE A. The report presents the implementation and recorded measurements for the linked 50,000-food development dataset.

1. Introduction and Dataset

The practical extension converts the project's command-line big-data workflow into a Flask-based web application named Food & Performance Explorer. It provides one interface for launching laptop development workflows and explaining saved MongoDB, Hadoop, Pig and Hive evidence.

1.1 Objectives

The dashboard aims to make each assignment theme understandable, display actual partial records beside calculated answers, show workflow progress, and provide commands that can verify the corresponding stored data. It keeps analysis controls distinct from result viewing so that previous results can be explored without automatically starting new jobs.

1.2 Dataset and experiment

Attribute	Reported value
Source	USDA FoodData Central, October 2024 CSV package
Food records analysed	50,000
Linked nutrient records	1,126,928
Nutrient definitions	477
Selection	First foods by source-row ordinal; all linked observations
Scope	Labelled development subset; one laptop with Docker workers

This report uses the successful 50,000-food run shown in the supplied dashboard evidence. The source includes more than two million food records; the displayed development results are not presented as measurements of the entire release.

1.3 Technology choice

The application uses Flask 3.1.2 with Python, HTML templates, CSS and JavaScript. MongoDB access uses PyMongo. Hadoop jobs, Pig transformations and Hive benchmarks are carried out by the existing processing tools; the dashboard presents and orchestrates their evidence rather than reimplementing their calculations in the browser.

2. Dashboard Architecture

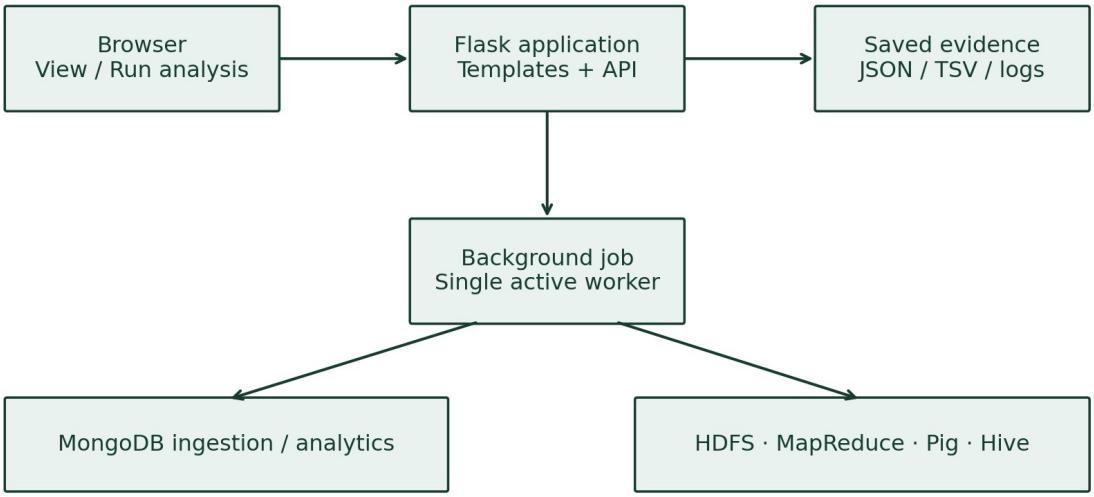


Figure 1. Flask result-viewing and background-execution architecture.

The browser communicates with Flask to render selected results and monitor permitted background actions. The backend reads saved JSON, TSV and log files, preserving the association between the selected run, its source fingerprint and its measurements.

2.1 Application interfaces

Interface	Purpose
GET /	Render the selected result or analysis controls.
GET /api/evidence	Return the selected run’s evidence.
GET /api/control/jobs	Read job states and recovery status.
POST /api/control/jobs	Start a predefined permitted workflow or service action.
GET /api/control/jobs/<id>/log	Read progress diagnostics for a job.

Saved result viewing remains available while computation services are stopped. Terminal verification commands inspect live storage and therefore require the relevant service to be running. Native college-cluster deployment uses view-only mode; the laptop-specific Docker controls are disabled in that mode.

3. Workflow Controls and Progress

3.1 Starting an analysis

The Run analysis tab fixes the source database as fdc_home202410 and offers development sizes, custom sizes from 1 to 100,000 foods, and a separately labelled full-release route. Every selected food includes its linked nutrient observations. New workflows use separate result identities so that saved runs remain available.

Stage	Action
1. Preflight	Check tools, project dependencies and Docker access.
2. MongoDB	Start MongoDB, select data, aggregate and profile indexes.
3. Export	Prepare the same linked records for Hadoop.
4. Cluster	Start Hadoop and wait for workers and Hive readiness.
5. Analytics	Run MapReduce, Pig and Hive; reconcile answers.
6. Evidence	Save counts, measurements, result files and completion state.

3.2 Progress and execution behaviour

The interface updates stage progress and exposes technical logs, including long-running Hive activity. Only one dashboard worker is allowed at a time. Closing the browser does not cancel the job. Service switching is handled by the workflow so MongoDB and Hadoop do not need to consume the laptop’s resources simultaneously.

3.3 Saved jobs and recovery

Current analysis status and previous job history are kept separate. An interrupted worker is identified explicitly; the Stop services and recover action stops project services before allowing a fresh job. Recovery retains existing datasets, results and logs. Saved success labels describe completed evidence, not a live service-health check.

4. Theme-Based Result Presentation

4.1 MongoDB: before, linking and after

The first theme shows a small preview of original food documents, observations linked to a preview food, derived food documents with nutrient arrays, and aggregation answers. The display then compares indexed and unindexed searches. Preview limits keep the interface readable without limiting the underlying calculations.

For the reported run, the preview includes WESSON Vegetable Oil, SWANSON BROTH BEEF and a CAMPBELL'S soup. The grouped branded_food protein result contains 32,840 observations, total amount 242,187.06 G and mean amount approximately 7.37476 G. The food-type and faceted outputs explain the group counts and overall coverage.

4.2 HDFS and MapReduce

The second theme presents the run's HDFS directory, first prepared nutrient records, verified file-management operations and a partial reducer result. It identifies two DataNodes and two YARN workers. A user can compare the displayed records with actual HDFS content by copying the adjacent read command.

4.3 Pig, Hive and comparison views

The third theme shows prepared, excluded and cleaned row counts, followed by the analytical answers shared by Text and RCFile tables. It separates query execution times from storage sizes. A filter can narrow the visible result sample by food type, nutrient or unit without changing the saved benchmark.

4.4 Source verification

Copy-command buttons provide read commands for MongoDB, HDFS, reducer output, Pig output and Hive tables. The page does not silently execute these commands. Older runs without previews are identified, and a preview-only command can capture examples from a matching database. Captures made after the analysis are labelled with that context.

5. Evaluation and Recorded Results

The supplied 50,000-food evidence shows completed analysis and matching cross-system counts, totals and means. Evaluation focuses on demonstrable source linkage, correct result presentation and accurate interpretation of the recorded performance metrics.

Check	Recorded result
Input and cleaning	1,126,928 prepared and cleaned observations; zero negatives excluded
MongoDB profiling	5 repetitions per plan; 50,000 vs 1 document examined
Median server time	42 ms collection scan; indexed value below 1 ms timer resolution
Hive answers	Matching results from Text and RCFile
Hive elapsed times	Nutrients: 66.24 / 64.22 s; protein: 64.91 / 67.35 s (Text / RCFile)

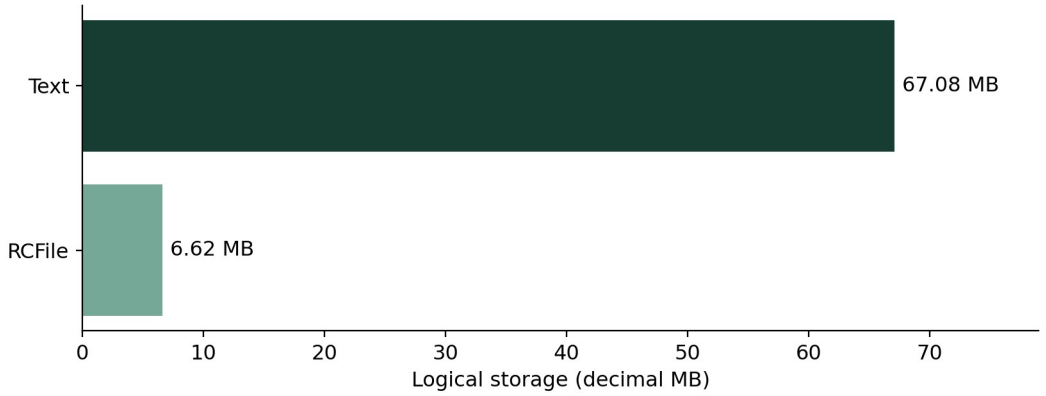


Figure 2. Recorded logical storage for the same cleaned input.

Text occupies 67,081,701 logical bytes and RCFile occupies 6,622,922 bytes, a reduction of approximately 90.13%. The chart uses decimal MB. Replicated sizes are twice the logical sizes in the recorded HDFS configuration. Query timing includes client startup, excludes warmups and has one measured repetition per question/table pair; no dashboard HTTP-latency benchmark is claimed.

6. Conclusion and References

Food & Performance Explorer supplies the Track B web-dashboard extension for the USDA analytics workflow. It combines workflow controls, progress visibility, saved evidence and theme-based explanations in a single interface. Before-and-after samples make the data linkage visible, and adjacent commands let a presenter verify the database and distributed files directly.

The reported development run covers 50,000 foods and all 1,126,928 linked nutrient records. The interface presents the measured index benefit and substantial RCFile storage reduction while preserving the distinction between saved output, current service state and benchmark timing. Its result-viewing mode also supports the project's native-cluster deployment route.

References

1. USDA FoodData Central — Download Datasets. https://fdc.nal.usda.gov/download-datasets/

2. Flask — Documentation. https://flask.palletsprojects.com/

3. MongoDB — Aggregation Pipelines. https://www.mongodb.com/docs/manual/core/aggregation-pipeline/

4. Apache Hadoop — Cluster Setup. https://apache.github.io/hadoop/hadoop-project-dist/hadoop-common/ClusterSetup.html

5. Apache Hive — Language Manual. https://cwiki.apache.org/confluence/display/Hive/LanguageManual

6. Project source code. https://github.com/Dhanush-singh-g/BDA-Assignment